

<https://tiny.pl/gkjsx88-vj>

KONSPEKT ZAJĘĆ LABORATORYJNYCH

LABORATORIUM 1

Środowisko pracy i przegląd narzędzi

Przedmiot:	Zaawansowany interfejs użytkownika		
Kierunek:	Informatyka II stopień, sem. 1	Forma:	Ćwiczenia laboratoryjne (LO)
Czas trwania:	90 minut (2 × 45 min)	ECTS:	3 pkt (za LO)
Prowadzący:	mgr inż. Dominik Aksamić	Rok akademicki:	2025/2026

1. Cele zajęć

Po zakończeniu laboratorium student:

- CEL 1 – WIEDZA: wymienia i charakteryzuje narzędzia ekosystemu front-endowego (Node.js, npm, VS Code, Git, Figma) oraz uzasadnia ich rolę w procesie wytwarzania interfejsów użytkownika.
- CEL 2 – UMIEJĘTNOŚCI KONFIGURACYJNE: samodzielnie instaluje i konfiguruje środowisko programistyczne (Node.js 20 LTS, npm, VS Code z zestawem 5 rozszerzeń) oraz weryfikuje poprawność konfiguracji komendami terminala.
- CEL 3 – UMIEJĘTNOŚCI PRAKTYCZNE: klonuje repozytorium Git, instaluje zależności projektu React (npm install), uruchamia serwer deweloperski (npm start) i obserwuje działającą aplikację w przeglądarce.
- CEL 4 – UMIEJĘTNOŚCI GIT: wprowadza zmianę w komponencie React (Header.jsx) i dokumentuje ją zgodnym z konwencją commitem (Conventional Commits), a następnie wypycha zmiany do zdalnego repozytorium.
- CEL 5 – KOMPETENCJE SPOŁECZNE: przestrzega standardów jakości kodu (ESLint, Prettier) i rozumie ich znaczenie dla pracy zespołowej w projekcie interfejsów użytkownika.

2. Wymagania wstępne dla studentów

Kursy poprzedzające: Projektowanie interfejsów użytkownika I i II, Grafika cyfrowa (zgodnie z sylabusem przedmiotu).

Student przystępujący do laboratorium powinien:

- posiadać podstawową znajomość HTML i CSS (selektory, box model, flexbox),

- rozumieć podstawy JavaScript (zmienne, funkcje, moduły ES6),
- znać podstawowe pojęcia systemu kontroli wersji Git (commit, branch, push, pull),
- dysponować komputerem z dostępem do Internetu i możliwością instalacji oprogramowania,
- posiadać konto na platformie GitHub (bezpłatne) – studenci zakładają konta przed zajęciami,
- mieć założone bezpłatne konto na platformie Figma (figma.com) – tworzenie konta trwa < 2 min.

Uwaga: Laboratorium nie zakłada wcześniejszej znajomości React.js. Framework zostanie omówiony na kolejnych zajęciach. Celem Lab 1 jest wyłącznie konfiguracja środowiska i weryfikacja jego działania na przykładzie prostego projektu startowego.

4. Instalacja i konfiguracja środowiska – instrukcja krok po kroku

4.1 Node.js 20 LTS

1. Przejdź na stronę <https://nodejs.org> i pobierz wersję oznaczoną LTS (Long Term Support). W momencie opracowania konspektu jest to Node.js 20.x.
2. Windows: uruchom pobrany instalator .msi i przejdź przez kreator, zaznaczając opcję "Automatically install the necessary tools".
3. macOS: użyj oficjalnego pakietu .pkg lub zainstaluj przez nvm (zalecane): `curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash`, następnie: `nvm install 20 && nvm use 20`.
4. Linux (Ubuntu/Debian): `curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash - && sudo apt-get install -y nodejs`
5. Weryfikacja instalacji – wpisz w terminalu: `node --version` (oczekiwany wynik: v20.x.x) oraz `npm --version` (oczekiwany wynik: 10.x.x).

4.2 Git

6. Windows: pobierz instalator z <https://git-scm.com> i wybierz opcję "Git Bash Here" w menu kontekstowym.
7. macOS: w terminalu wpisz `git --version` – jeśli Xcode CLT nie są zainstalowane, system automatycznie zaoferuje instalację. Alternatywnie: `brew install git`.
8. Linux: `sudo apt-get install git` (Debian/Ubuntu) lub `sudo dnf install git` (Fedora/RHEL).
9. Konfiguracja globalna (wymagana przed pierwszym commitem): `git config --global user.name "Imię Nazwisko"` oraz `git config --global user.email "adres@email.com"`.
10. Weryfikacja: `git --version` (oczekiwany wynik: git version 2.x.x).

4.3 Visual Studio Code

11. Pobierz instalator z <https://code.visualstudio.com> odpowiedni dla Twojego systemu operacyjnego.
12. Windows: zaznacz podczas instalacji opcje "Add to PATH" i "Open with Code" w menu kontekstowym.
13. macOS: po zainstalowaniu aplikacji otwórz VS Code, uruchom paletę poleceń (Cmd+Shift+P) i wpisz Shell Command: Install 'code' command in PATH.
14. Linux: zainstaluj pakiet .deb lub .rpm z oficjalnej strony. Alternatywnie przez snap: `sudo snap install --classic code`.
15. Weryfikacja: w terminalu wpisz `code --version`.

4.4 Rozszerzenia VS Code

Zainstaluj każde rozszerzenie przez Ctrl+Shift+X (panel Extensions) lub poleceniem w terminalu:

- ESLint (dbaeumer.vscode-eslint) – analiza statyczna kodu JavaScript/TypeScript. Polecenie: `code --install-extension dbaeumer.vscode-eslint`
- Prettier – Code formatter (esbenp.prettier-vscode) – formatowanie kodu. Polecenie: `code --install-extension esbenp.prettier-vscode`
- GitLens — Git supercharged (eamodio.gitlens) – zaawansowana integracja z Git (blame, historia linii). Polecenie: `code --install-extension eamodio.gitlens`
- vscode-styled-components (styled-components.vscode-styled-components) – podświetlanie składni dla Styled Components. Polecenie: `code --install-extension styled-components.vscode-styled-components`
- Figma for VS Code (figma.figma-vscode-extension) – podgląd zasobów Figma w edytorze. Polecenie: `code --install-extension figma.figma-vscode-extension`

Konfiguracja ustawień VS Code – utwórz plik `.vscode/settings.json` w projekcie:

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "eslint.validate": ["javascript", "javascriptreact"],
  "gitlens.hovers.currentLine.over": "line"
}
```

5. Zadanie weryfikacyjne

Tytuł zadania: Sklonuj repozytorium startowe, uruchom projekt React, zmień komponent Header i wykonaj commit.

Zadanie wykonuje się indywidualnie. Czas wykonania: 13 minut (blok 65–78 min wg planu zajęć). Prowadzący obserwuje pracę studentów i udziela wskazówek.

Krok 1 – Klonowanie repozytorium

16. Otwórz terminal (VS Code Terminal lub systemowy).

17. Przejdź do katalogu, w którym chcesz przechowywać projekt: `cd ~/Documents/studia`
18. Sklonuj repozytorium startowe (prowadzący podaje adres URL podczas zajęć): `git clone https://github.com/<organizacja>/advanced-ui-starter.git`
19. Wejdź do katalogu projektu: `cd advanced-ui-starter`

Krok 2 – Instalacja zależności i uruchomienie

20. Zainstaluj wszystkie zależności: `npm install` (oczekiwany czas: 30–60 sekund).
21. Uruchom serwer deweloperski: `npm start`
22. Otwórz przeglądarkę pod adresem `http://localhost:3000` – powinna wyświetlić się strona startowa z nagłówkiem "Advanced UI – Starter".

Krok 3 – Modyfikacja komponentu Header

23. Otwórz plik `src/components/Header.jsx` w VS Code.
24. Zlokalizuj element `<h1>` w komponencie.
25. Zmień tekst wewnątrz znacznika `<h1>` na: `"<Twoje Imię i Nazwisko> – Lab 1"` (np. Jan Kowalski – Lab 1).
26. Zapisz plik (`Ctrl+S` / `Cmd+S`) – strona powinna odświeżyć się automatycznie (Hot Module Replacement).
27. Sprawdź w przeglądarce, że nagłówek wyświetla Twoje imię i nazwisko.

Krok 4 – Commit i push

28. Sprawdź status repozytorium: `git status` (powinien wyświetlić zmodyfikowany plik `Header.jsx`).
29. Dodaj plik do staging area: `git add src/components/Header.jsx`
30. Utwórz commit z opisową wiadomością zgodną z Conventional Commits: `git commit -m "feat(header): update title with student name for Lab 1 verification"`
31. Wypchnij zmiany na zdalne repozytorium (branch: `main` lub przypisany branch studenta): `git push origin main`
32. Zweryfikuj na GitHub, że Twój commit jest widoczny w historii repozytorium.

6. Szablon repozytorium startowego

6.1 Struktura plików

Ścieżka / Plik	Opis
advanced-ui-starter/	Katalog główny projektu
├─ public/	Statyczne zasoby (index.html, favicon)

Ścieżka / Plik	Opis
└─ index.html	Punkt wejścia HTML, <div id='root'>
└─ src/	Kod źródłowy aplikacji React
└─ components/	Komponenty wielokrotnego użytku
└─ Header.jsx	Komponent nagłówka – cel zadania weryfikacyjnego
└─ Footer.jsx	Komponent stopki
└─ Button.jsx	Przykładowy komponent przycisku
└─ pages/	Strony / widoki aplikacji
└─ Home.jsx	Strona główna
└─ styles/	Pliki stylów globalnych i komponentowych
└─ global.css	Reset CSS i zmienne
└─ App.jsx	Główny komponent, routing
└─ index.js	Punkt wejścia React (ReactDOM.render)
└─ .eslintrc.json	Konfiguracja ESLint (airbnb-base + react)
└─ .prettierrc	Konfiguracja Prettier
└─ .gitignore	node_modules, build, .env
└─ package.json	Zależności i skrypty npm
└─ README.md	Instrukcja uruchomienia i opis projektu

6.2 Zawartość package.json

```
{
  "name": "advanced-ui-starter",
  "version": "1.0.0",
  "description": "Projekt startowy - Zaawansowany interfejs użytkownika",
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "lint": "eslint src --ext .js,.jsx",
    "format": "prettier --write src/**/*.{js,jsx,css}"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-scripts": "5.0.1",
    "styled-components": "^6.1.13"
  },
  "devDependencies": {
    "eslint": "^8.57.0",
    "eslint-config-react-app": "^7.0.1",
    "prettier": "^3.3.3"
  },
  "eslintConfig": {
```

```

    "extends": ["react-app", "react-app/jest"]
  },
  "browserslist": {
    "production": [">0.2%", "not dead", "not op_mini all"],
    "development": ["last 1 chrome version", "last 1 firefox version"]
  }
}

```

7. Kryteria oceny i punktacja

Kryterium	Punkty	Uwagi
Środowisko skonfigurowane poprawnie (Node.js, VS Code, Git)	0–2 pkt	Weryfikacja komendami terminala
Wszystkie 5 rozszerzeń zainstalowane i aktywne	0–2 pkt	Sprawdzenie listy rozszerzeń
Repozytorium sklonowane, projekt uruchamia się (npm start)	0–2 pkt	Localhost widoczny w przeglądarce
Poprawna modyfikacja komponentu Header	0–2 pkt	Zmiana widoczna w przeglądarce
Commit z opisową wiadomością, push do repo	0–2 pkt	git log zawiera commit studenta

Skala ocen: 0–3 pkt → 2,0 | 4–5 pkt → 3,0 | 6–7 pkt → 3,5 | 8 pkt → 4,0 | 9 pkt → 4,5 | 10 pkt → 5,0

Ocena częściowa z laboratorium jest uwzględniana w ocenie końcowej jako składnik oceny aktywności (wg sylabusu przedmiotu). Minimalna liczba punktów do uzyskania zaliczenia laboratorium 1: 4 pkt (warunek konieczny: działające środowisko oraz uruchomiony projekt).

8. FAQ – najczęstsze problemy i rozwiązania

System	Problem	Przyczyna	Rozwiązanie
Windows	node nie jest rozpoznawany po instalacji	PATH nie został odświeżony	Zamknij i otwórz terminal ponownie lub uruchom: refreshenv (Chocolatey)
Windows	npm install EACCES / EPERM	Brak uprawnień do katalogu	Uruchom terminal jako Administrator lub zmień katalog globalny npm

System	Problem	Przyczyna	Rozwiązanie
Windows	git bash vs PowerShell – różne zachowanie PATH	Różne środowiska shell	Używaj Git Bash lub ustaw PATH systemowo. Zalecane: Windows Terminal + PowerShell 7
Mac	node: command not found (po instalacji nvm)	nvm nie załadowany w shellu	Dodaj do ~/.zshrc: export NVM_DIR i source nvm.sh, następnie: source ~/.zshrc
Mac	xcrun: error – Xcode Command Line Tools	Brak Xcode CLT wymaganych przez node-gyp	Uruchom: xcode-select --install
Mac (M1/M2/M3)	Błędy przy npm install – architektura arm64	Pakiet nie wspiera arm64	Użyj: arch -x86_64 npm install lub zaktualizuj pakiet do nowszej wersji
Linux	npm EACCES przy instalacji globalnej	Brak uprawnień do /usr/lib/node_modules	Zmień katalog globalny: mkdir ~/.npm-global && npm config set prefix '~/.npm-global' i dodaj do PATH
Linux	VS Code nie uruchamia się – Sandbox error	Brakuje sandbox flag (np. Wayland)	Uruchom: code --no-sandbox lub dodaj --disable-gpu
Wszystkie	Port 3000 jest zajęty – npm start fail	Inny proces używa portu 3000	Uruchom na innym porcie: PORT=3001 npm start lub zabij proces: lsof -ti:3000 xargs kill
Wszystkie	ESLint nie podkreśla błędów w VS Code	Brak lokalnej instalacji ESLint lub zła wersja rozszerzenia	Uruchom w projekcie: npm install --save-dev eslint i przeładuj VS Code (Ctrl+Shift+P > Reload Window)

Wskazówka ogólna: Przed zgłoszeniem problemu prowadzącemu zawsze spróbuj: (1) zamknąć i otworzyć terminal, (2) sprawdzić zmienne PATH, (3) uruchomić VS Code z prawami administratora (Windows). Większość problemów z instalacją wynika z braku odświeżenia sesji terminala po zmianach PATH.